

Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación

TA-IND-01 — Informe Académico Individual

Análisis Comparativo de Arquitecturas Distribuidas:

Microservicios vs. Monolito Modular

Nombre: Jhinson Stalyn Aucatoma Celorio
Curso: 7mo Software “A”
Asignatura: Aplicaciones Distribuidas
Docente: Ing. Guerrero Ulloa Gleiston Cicerón
Fecha: Sábado 13 de junio del 2026

[Presione este texto para ir al repositorio de la actividad](#)

Abstract

The present report analyzes and compares two of the main architectural styles used in the development of distributed systems: monolithic architecture and microservices architecture. To this end, recent academic sources addressing their definitions, components, advantages, and limitations were reviewed.

The comparison was carried out based on technical criteria: scalability, coupling, deployment, maintenance, and operational complexity. The results indicate that monolithic architecture offers greater simplicity for small-scale projects with limited scope, whereas microservices architecture is more suitable for complex systems that require high availability, independent scaling, and autonomous development teams.

In conclusion, the choice between these two architectures is not absolute, as it depends on the context, available resources, and project objectives. For the team’s Final Course Project, the adoption of a microservices architecture is justified due to the functional scope of the system and the need to scale components independently.

Resumen

El presente informe analiza y compara dos de los principales estilos arquitectónicos utilizados en el desarrollo de sistemas distribuidos: la arquitectura monolítica y la arquitectura de microservicios. Para ello, se revisaron fuentes académicas recientes que abordan sus definiciones, componentes, ventajas y limitaciones.

La comparación se realizó en base a criterios técnicos: escalabilidad, acoplamiento, despliegue, mantenimiento y complejidad operacional. Los resultados indican que la arquitectura monolítica ofrece mayor simplicidad en proyectos pequeños y de alcance limitado, mientras que la arquitectura de microservicios resulta más adecuada para sistemas complejos que requieren alta disponibilidad, escalado independiente y equipos de desarrollo autónomos.

Como conclusión, la elección entre ambas arquitecturas no es absoluta ya que depende del contexto, los recursos disponibles y los objetivos del proyecto. Para el Proyecto Final de Curso del equipo, se justifica la adopción de una arquitectura de microservicios debido al alcance funcional del sistema y la necesidad de escalar componentes de forma independiente.

1. Introducción

¿Qué son las arquitecturas distribuidas?

Los sistemas distribuidos describen una clase de sistema computacional en el que componentes de hardware y software están conectados mediante una red, y coordinan sus acciones a través del paso de mensajes para alcanzar un objetivo común [1]. En términos más simples, en lugar de tener todo el procesamiento en una sola máquina, el trabajo se reparte entre varias computadoras que se comunican entre sí. Un sistema distribuido es aquel en el que los componentes o recursos están distribuidos en una red en diferentes ubicaciones e interactúan entre sí solo mediante el paso de mensajes, apareciendo como un sistema coherente único para sus usuarios [2].

Importancia actual

Los sistemas distribuidos han sido un campo activo de investigación durante más de 60 años y han tenido un papel muy importante en las ciencias de la computación, permitiendo la invención del Internet que sustenta todos los aspectos de la vida moderna [1]. Con el paso del tiempo, estos sistemas han evolucionado considerablemente: a través de los avances tecnológicos y su cambiante rol en la sociedad, han experimentado una evolución constante, con cada cambio resultando en la formación de un nuevo paradigma. Entre estos paradigmas hoy destacan la computación en la nube, la computación Fog y el Internet de las Cosas (IoT), cada uno de los cuales permite nuevas formas de valor comercial, aunque también trae consigo nuevos desafíos que deben ser abordados [1].

Objetivo del informe

Este informe tiene como objetivo analizar las principales arquitecturas utilizadas en el desarrollo de sistemas distribuidos: la arquitectura monolítica y la arquitectura de microservicios, comparando sus características, ventajas y desventajas, con el fin de determinar cuál de ellas resulta más adecuada para el desarrollo del Proyecto Final de Curso (PFC).

2. Marco Teórico

2.1. Arquitectura Monolítica

Definición

La arquitectura monolítica describe un sistema de software diseñado, implementado, empaquetado y desplegado como una sola unidad. Todos los subsistemas se comunican internamente, en lugar de hacerlo a través de la red, y todos los datos se almacenan en una única base de datos o nodo de almacenamiento, siendo accesibles externamente mediante las API que expone el sistema. Cualquier cambio en el código, ya sea para corregir un fallo o para ampliar la funcionalidad, requiere la reconstrucción y el despliegue completo de la aplicación [3].

Una aplicación monolítica se caracteriza por tener un único archivo ejecutable, aunque puede estar compuesta por varios módulos que no pueden ejecutarse de forma independiente. Si bien una aplicación puede dividirse en varias capas lógicas, el despliegue de todas ellas se lleva a

cabo conjuntamente como un único bloque, generando un único proceso en ejecución. Generalmente, este tipo de sistema utiliza una sola base de código para exponer todos sus servicios o funcionalidades [4].

Con el tiempo, la arquitectura monolítica puede volverse difícil de mantener, ya que los cambios en la lógica de un módulo tienden a afectar el código de otros, lo que dificulta preservar una estructura modular clara [5]. Entonces, una aplicación monolítica es aquella que combina todos (o muchos de) sus módulos en componentes autocontenidos e independientes de otras aplicaciones externas [6].

Componentes

La arquitectura de referencia para aplicaciones monolíticas es la de una aplicación con divisiones en varias capas lógicas. El patrón más utilizado es el de tres capas, en el que una aplicación web se organiza de la siguiente manera [4]:

Cuadro 1: Capas lógicas del patrón de tres niveles en una arquitectura monolítica [4], [7].

Capa	Descripción
Presentación	Contiene la lógica para responder a las interacciones del usuario con la aplicación a través de un navegador web (páginas HTML y JavaScript).
Negocio	Alberga las reglas y validaciones de las funcionalidades del negocio; gestiona las solicitudes HTTP, ejecuta la lógica de dominio y selecciona las vistas HTML a enviar.
Acceso a Datos	Incluye todos los mecanismos para realizar llamadas tanto a bases de datos como a servicios externos que la aplicación necesita. Recupera y actualiza los datos de la base de datos.

Esto se puede ver representado de mejor manera en la Figura 1 [4]:

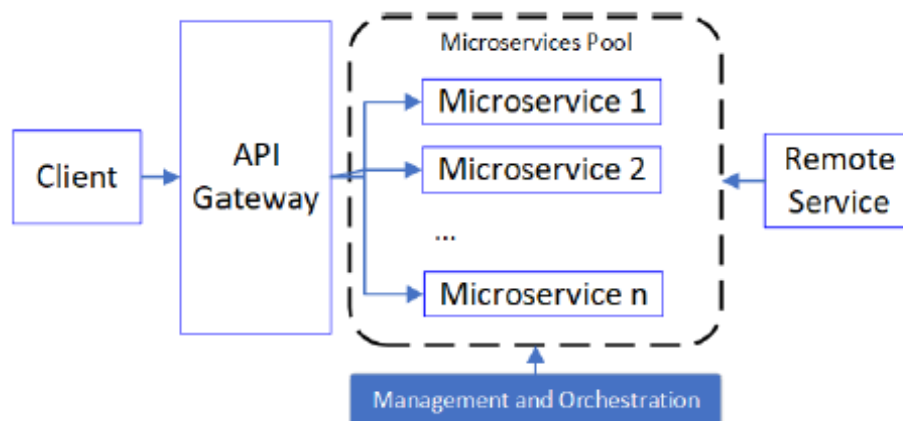


Figura 1: Estructura general de la arquitectura monolítica.

Ventajas

La arquitectura monolítica ofrece importantes ventajas, especialmente cuando la aplicación es pequeña o está en sus primeras etapas de desarrollo. A continuación, se describen las principales:

Cuadro 2: Principales ventajas de la arquitectura monolítica.

Ventaja	Descripción
Consistencia de datos	Al centralizar todos los datos en una sola base de datos, no se requiere sincronización entre servicios y se garantiza la consistencia durante la ejecución [3], [7].
Simplicidad	Las aplicaciones monolíticas son más fáciles de probar, implementar, depurar y monitorear. Un desarrollador puede rastrear el código fuente de principio a fin rápidamente [4], [7].
Rendimiento de comunicación	La comunicación entre subsistemas es completamente interna (en memoria), sin llamadas remotas, lo que elimina la sobrecarga de red. El tiempo de respuesta promedio medido experimentalmente fue de 1,12 ms [5], [7].
Seguridad	Un único nodo de procesamiento y una única fuente de datos reducen las posibles vulnerabilidades de red, a diferencia de los microservicios donde cada servicio adicional puede introducir nuevos riesgos [3].
Reutilización de código	Es posible reutilizar porciones de código dentro de la misma aplicación con control total sobre los cambios, algo más sencillo que en sistemas distribuidos donde una modificación puede afectar a otros consumidores [4].
Mejor rendimiento en una sola máquina	Los resultados experimentales indican que en una sola máquina, un monolito tiene mejor rendimiento que una aplicación basada en microservicios, debido a la ausencia de sobrecarga de comunicación entre servicios [7].

Desventajas

A medida que la aplicación crece en tamaño y complejidad, la arquitectura monolítica comienza a presentar limitaciones importantes:

Cuadro 3: Principales desventajas de la arquitectura monolítica.

Desventaja	Descripción
Alto costo de mantenimiento	En sistemas grandes, identificar un error, corregirlo y desplegarlo puede ser costoso y lento. Cualquier cambio en un módulo puede provocar fallos en toda la aplicación y obliga a volver a probar todas las funcionalidades [4].
Escalabilidad limitada	Si un módulo específico requiere más recursos, no es posible escalarlo de forma aislada: debe escalarse toda la aplicación, lo que incrementa costos de hardware, software y licencias [4], [5].
Bloqueo tecnológico	Por lo general, las aplicaciones monolíticas están limitadas a un único lenguaje de programación, a diferencia de los microservicios donde cada servicio puede desarrollarse con tecnologías diferentes [4].
Agilidad reducida	Actualizar el código lleva más tiempo y requiere comprobaciones exhaustivas antes de cada despliegue. El crecimiento del monolito incrementa el tiempo de arranque y dificulta la integración y entrega continua (CI/CD) [5], [7].
Problemas organizacionales	El aumento del número de desarrolladores trabajando sobre una misma base de código puede producir una utilización desigual del personal y pérdidas de productividad [7].
Escalado horizontal ineficiente	Para incrementar la disponibilidad es necesario replicar toda la aplicación, incluso los servicios menos utilizados, lo que genera una utilización subóptima de recursos [6], [7].

2.2. Arquitectura de Microservicios

Definición

La arquitectura de microservicios divide la aplicación en pequeños servicios autónomos que se comunican a través de la red. Cada microservicio solo tiene acceso a los datos necesarios para su propio funcionamiento, por lo que los datos de la aplicación se distribuyen en bases de datos independientes por servicio. Dado que la MSA se caracteriza por una clara separación de responsabilidades, la corrección de un error o un cambio de funcionalidad solo requiere la reconstrucción y el despliegue del microservicio responsable [3].

Un microservicio es un elemento de una aplicación que puede ejecutarse de forma independiente. A diferencia del enfoque monolítico, cada microservicio se desarrolla para cumplir una funcionalidad empresarial específica, tiene un acoplamiento flexible, una alta cohesión funcional y puede interactuar con otros microservicios a través de un sistema de mensajería [4]. La comunicación entre servicios se implementa generalmente mediante HTTP/HTTPS, WebSockets u otros protocolos ligeros [5].

Las arquitecturas de microservicios se construyen sobre los principios de responsabilidad única, alta cohesión, bajo acoplamiento, escalabilidad, capacidad de despliegue independiente y mínima alteración a otros microservicios que componen el sistema. Cada servicio puede diseñarse, implementarse, probarse, desplegarse y escalarse de forma autónoma, incluyendo la posibilidad

de utilizar diferentes pilas tecnológicas [\[6\]](#), [\[8\]](#).

Componentes

La arquitectura de microservicios cuenta con varios componentes que trabajan en conjunto para garantizar el funcionamiento autónomo de cada servicio. A continuación, se describen los principales componentes según Richards y Ford [\[9\]](#):

Cuadro 4: Componentes principales de la arquitectura de microservicios según Richards y Ford [9].

Componente	Descripción
API Layer (Capa de API)	Componente opcional situado entre los consumidores (interfaces de usuario u otros sistemas) y los servicios. Sirve como punto de indirección o proxy para funciones operativas como el naming. No debe usarse como orquestador, ya que toda la lógica relevante debe residir dentro del bounded context de cada servicio [9].
Servicios	Componente central de la arquitectura. Cada servicio modela un dominio o subdominio (bounded context) y debe incluir todas las partes necesarias para operar de forma independiente, incluyendo su propia base de datos y módulos internos [9].
Base de datos por servicio	Cada servicio dispone de su propia base de datos para lograr aislamiento de datos, evitando esquemas compartidos que generen dependencias entre servicios [9].
Sidecar	Componente incluido en cada servicio para manejar preocupaciones operativas comunes: circuit breaker, logging, monitoring, entre otras. Puede ser gestionado por los equipos individuales o por un equipo de infraestructura compartida [9].
Service Mesh / Service Plane	Se forma por la conexión de todos los sidecars. Provee una interfaz operativa consistente entre todos los microservicios, permitiendo control unificado sobre logging, monitoreo y otras preocupaciones transversales [9].
Service Discovery	Herramienta que monitorea el número y frecuencia de solicitudes y puede crear nuevas instancias de servicios para manejar escala o elasticidad. Generalmente se incluye dentro del service mesh y se aloja en la API layer [9].
Frontends	Existen dos patrones comunes: el frontend monolítico (una sola interfaz que llama a través de la API layer) y los microfrontends (componentes de interfaz emitidos por cada servicio individual, coordinados para formar la interfaz completa) [9].
Mediador / Orquestador	Servicio cuya única responsabilidad es coordinar llamadas a otros servicios para procesos de negocio complejos. En el patrón Saga, actúa como mediador en transacciones distribuidas, registrando éxito o fallo de cada parte y ejecutando transacciones compensatorias en caso de error [9].

La Figura 2 (extraída de Velepucha y Flores [4]) ilustra la estructura general de la arquitectura de microservicios con sus capas de comunicación, y la Figura 3 (extraída de Richards y Ford [9]) ofrece una vista más detallada de los componentes descritos anteriormente.

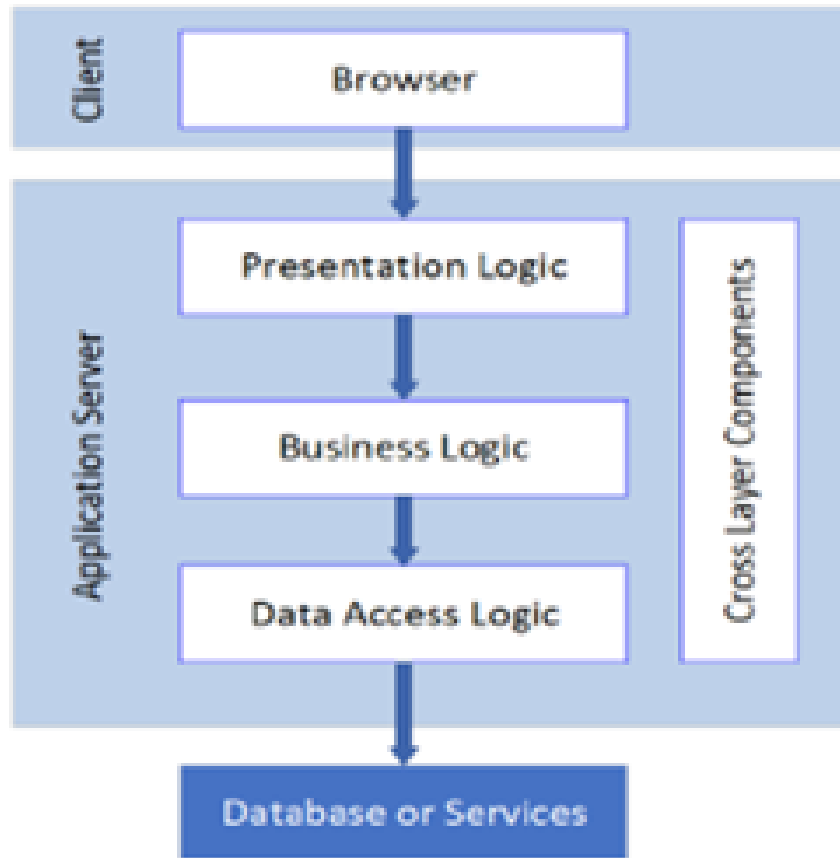


Figura 2: Estructura general de la arquitectura de microservicios.

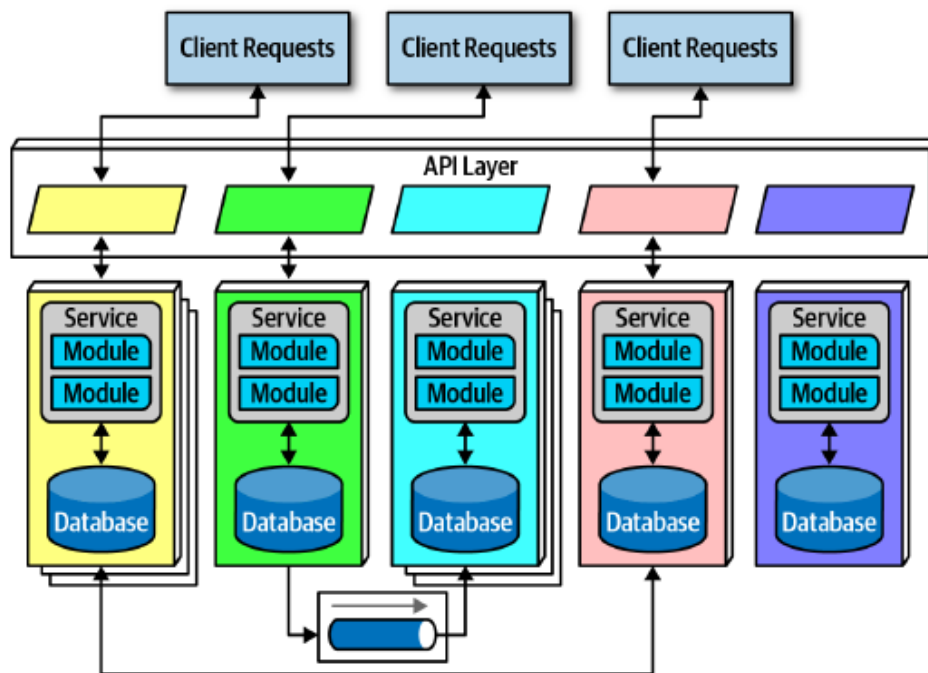


Figura 3: Estructura detallada de la arquitectura de microservicios.

Ventajas

Las principales ventajas de los microservicios radican en su capacidad para aprovechar los beneficios de la computación en la nube y los principios de desarrollo ágil [3]:

Cuadro 5: Principales ventajas de la arquitectura de microservicios.

Ventaja		Descripción
Alta disponibilidad		Al lanzar nuevas versiones de un microservicio no es necesario detener todo el sistema. Las nuevas solicitudes se dirigen automáticamente a la nueva versión, mientras los microservicios existentes continúan atendiendo peticiones hasta que finalicen. En caso de fallos, solo el microservicio afectado deja de funcionar, sin impactar al resto del sistema [4].
Mantenimiento simplificado		Al ser componentes pequeños e independientes, es más fácil navegar por el código, encontrar errores y corregirlos sin afectar a otros módulos. Esto facilita la publicación de nuevas versiones [4].
Seguridad y auditoría granular		Es posible implementar mecanismos de seguridad y monitoreo específicos en microservicios individuales, permitiendo auditorías más precisas y optimizando los costos de seguridad [4].
Pruebas independientes		Los microservicios no se afectan entre sí al ser probados de forma aislada. Al combinarse con prácticas DevOps y pruebas automatizadas integradas en el proceso de compilación, aumentan las probabilidades de detectar y corregir errores tempranamente [4].
Equipos autónomos		Cada equipo puede tomar uno o más microservicios y gestionarlos de forma completamente autónoma, incluyendo el diseño, el desarrollo y el despliegue. Los equipos pueden estar ubicados en distintas localizaciones geográficas y trabajar en paralelo sin interferirse [4].
Despliegue independiente y CI/CD		Una de las ventajas más destacadas es la posibilidad de desplegar cada servicio de forma independiente, lo que facilita la orquestación y permite realizar cambios o actualizaciones sin afectar a otros microservicios. El uso de herramientas CI/CD en cada microservicio agiliza las tareas de desarrollo y mantenimiento futuras [10].

Desventajas

Sin embargo, la arquitectura de microservicios también presenta desventajas:

Cuadro 6: Principales desventajas de la arquitectura de microservicios.

Desventaja	Descripción
Curva de aprendizaje	Las organizaciones que adoptan microservicios por primera vez enfrentan desafíos significativos derivados de la experiencia limitada del equipo. Un estudio de caso indica que una migración requirió ocho meses para superar la curva de aprendizaje y operar correctamente con esta arquitectura [4].
Sobrecarga de red	Los microservicios realizan un mayor número de llamadas a través de la red, lo que provoca retrasos en la comunicación y sobrecarga. Esto puede requerir implementar llamadas asíncronas y procesamiento paralelo para compensar [4].
Duplicación de datos y consistencia	Dado que cada microservicio gestiona su propia base de datos, los datos pueden duplicarse entre servicios. Garantizar la consistencia requiere mecanismos adicionales de programación. Además, no se recomienda el uso de transacciones distribuidas, por lo que deben desarrollarse servicios compensatorios como alternativa [4].
Mayor complejidad operacional	Gestionar muchos microservicios simultáneamente, incluyendo el rastreo de errores a través de la trazabilidad puede ser complejo y consumir muchas horas de trabajo. Sin automatización (DevOps), operar esta arquitectura se vuelve inmanejable [4].
Separación de datos y latencia	La distribución de datos puede provocar inconsistencias y problemas de acceso entre sistemas o usuarios finales. La sobrecarga de comunicaciones derivada de la red puede generar latencia y efectos secundarios no deseados en la funcionalidad esperada [3].

3. Análisis Comparativo

La siguiente tabla muestra las diferencias entre ambas arquitecturas en base a los criterios técnicos más relevantes:

Cuadro 7: Comparación entre arquitectura monolítica y arquitectura de microservicios.

Criterio	Monolítico	Microservicios
Escalabilidad	Escalado vertical (más CPU/-RAM) o replicación de toda la aplicación. El escalado vertical duplica aproximadamente el rendimiento, pero está limitado por el costo y la disponibilidad de máquinas más potentes [7], [8].	Cada servicio puede escalarse de forma independiente según la demanda, lo que resulta más eficiente y económico. Se construye sobre el principio de escalabilidad independiente [6], [8].
Acoplamiento	Los módulos tienden a ser altamente acoplados y poco cohesivos. Esta interdependencia es una de las limitaciones que motivan la migración hacia microservicios [6].	Bajo acoplamiento y alta cohesión son principios fundacionales. Cada servicio es independiente y puede evolucionar sin impactar a los demás. Las métricas de acoplamiento son clave para evaluar la calidad del diseño [6].
Despliegue	Se despliega como una única unidad. Cualquier cambio, por pequeño que sea, puede requerir reintegrar, reconstruir, volver a probar y reiniciar toda la aplicación [6], [7].	Cada servicio se despliega de forma independiente. Esto permite actualizaciones frecuentes sin afectar al resto del sistema [8], [10].
Mantenimiento	A medida que el sistema crece, mantener el código y aislar cambios en módulos específicos se vuelve progresivamente más difícil [6], [7].	Los componentes pueden evolucionar y desplegarse de forma independiente, lo que facilita el mantenimiento a largo plazo. Sin embargo, la complejidad de coordinar múltiples servicios puede ser un desafío [6], [10].
Complejidad	Menor complejidad de infraestructura ya que no requiere gateway, descubrimiento de servicios ni configuración distribuida. Sin embargo, la complejidad interna del código aumenta con el tiempo [8].	Mayor complejidad operacional: requiere gestionar comunicaciones entre servicios, transacciones distribuidas y pruebas de integración. La infraestructura de soporte (gateway, service discovery, configuración) añade capas adicionales [8].
Rendimiento	Mayor rendimiento en una sola máquina gracias a la comunicación interna en memoria. Tiempo de respuesta promedio: $\sim 1,12$ ms [5], [7].	La comunicación entre servicios introduce latencia de red. Sin embargo, el escalado independiente permite manejar mayor volumen de solicitudes distribuidas [5], [8].
Tecnología	Generalmente limitado a un único lenguaje de programación y pila tecnológica [4].	Cada servicio puede desarrollarse con el lenguaje y la tecnología más adecuados para su función específica [4], [8].

4. Aplicación al Proyecto Final de Curso (PFC)

Para el desarrollo de **TiendaTech**, una plataforma de venta de componentes de computadora que incluye funcionalidades como gestión de usuarios, catálogo de productos, carrito de compras, pagos, facturación, ensamblado personalizado de computadoras y un asistente inteligente, se ha seleccionado una arquitectura de microservicios. Esta decisión se debe a que cada módulo del sistema posee responsabilidades específicas y puede evolucionar de forma independiente. La separación en microservicios permite organizar mejor el desarrollo, facilitar el mantenimiento del software y escalar únicamente aquellos componentes que presenten una mayor demanda, como el catálogo de productos, las búsquedas o el asistente inteligente, optimizando así el uso de recursos.

Además, esta arquitectura favorece que los distintos integrantes del equipo trabajen simultáneamente en diferentes módulos sin generar dependencias excesivas entre ellos. Entre los beneficios esperados se encuentran una mayor disponibilidad del sistema, despliegues independientes, facilidad para incorporar nuevas funcionalidades y una mejor capacidad de crecimiento a futuro. Aunque la implementación de microservicios introduce desafíos relacionados con la comunicación entre servicios, la gestión de datos distribuidos y la complejidad de la infraestructura, estas dificultades son compensadas por la flexibilidad y escalabilidad que requiere un sistema como TiendaTech, cuyo alcance integra múltiples procesos de negocio claramente diferenciados.

5. Discusión

La arquitectura monolítica, a pesar de su simplicidad inicial, presenta una limitación fundamental: su dificultad para adaptarse al crecimiento. A medida que el sistema escala en funcionalidad y en carga, los tiempos de arranque aumentan, los despliegues se vuelven más riesgosos y la base de código se vuelve progresivamente más difícil de mantener [6], [7]. Esta rigidez puede frenar la capacidad de innovación de los equipos de desarrollo.

Por su parte, la arquitectura de microservicios enfrenta limitaciones en sus etapas iniciales: su implementación correcta requiere una infraestructura más sofisticada, mayor experiencia del equipo y una cultura organizacional orientada a la automatización y la colaboración. La gestión de transacciones distribuidas, la trazabilidad de errores entre servicios y la consistencia eventual de los datos son desafíos técnicos que no deben subestimarse [3], [4].

El monolito supera a los microservicios en escenarios donde el sistema es pequeño o mediano, el equipo cuenta con pocos desarrolladores, el presupuesto de infraestructura es limitado, o cuando el proyecto se encuentra en una fase de validación de su modelo de negocio. En estos contextos, la velocidad de desarrollo y la simplicidad operacional son más valiosas que la escalabilidad [3], [5].

Los microservicios superan al monolito cuando el sistema debe soportar alta concurrencia de usuarios, cuando diferentes módulos tienen requisitos de rendimiento muy distintos, cuando el equipo es grande y distribuido, o cuando la continuidad del servicio es crítica para el negocio. También son preferibles cuando se requiere flexibilidad tecnológica o cuando diferentes partes del sistema evolucionan a distintas velocidades [6], [8], [10].

6. Conclusiones

El análisis realizado permitió identificar que la arquitectura monolítica y la arquitectura de microservicios responden a necesidades diferentes. Mientras que el monolito destaca por su simplicidad y facilidad de implementación en proyectos pequeños o con recursos limitados, los microservicios ofrecen mayores capacidades de escalabilidad, mantenimiento y evolución para sistemas más complejos [3], [6], [7].

Asimismo, se concluye que no existe una arquitectura universalmente mejor, ya que la elección depende de factores como el tamaño del equipo, los requerimientos del sistema y los recursos

disponibles [3], [5]. En el caso de TiendaTech, la arquitectura de microservicios resulta la alternativa más adecuada debido a la variedad de módulos que componen el sistema, la posibilidad de desarrollar servicios de forma independiente y la necesidad de permitir un crecimiento futuro de la plataforma. Aunque esta arquitectura implica desafíos técnicos adicionales, sus beneficios a largo plazo justifican su adopción para el proyecto [4], [8], [10].

Declaración de uso de inteligencia artificial

Durante la elaboración de este informe se utilizó la herramienta de inteligencia artificial Claude como apoyo para la revisión de la redacción, mejora de la coherencia entre ideas y adecuación de la terminología técnica empleada. La búsqueda, selección y análisis de las fuentes bibliográficas, así como las conclusiones y criterios expuestos en el documento, fueron realizados por el autor. El uso de la herramienta solo se limitó para cuidar la calidad de la escritura y la organización del contenido.

Referencias

- [1] D. Lindsay, S. S. Gill, D. Smirnova y P. Garraghan, «The evolution of distributed computing systems: from fundamental to new frontiers,» *Computing*, vol. 103, n.º 8, págs. 1859-1878, 2021. DOI: [10.1007/s00607-020-00900-y](https://doi.org/10.1007/s00607-020-00900-y).
- [2] M. M. AlShammari, «Evolution of Issues in Distributed Systems: A Systematic Review,» en *2020 19th International Symposium on Distributed Computing and Applications for Business Engineering and Science (DCABES)*, Xuzhou, China, 2020. DOI: [10.1109/DCABES50732.2020.00018](https://doi.org/10.1109/DCABES50732.2020.00018).
- [3] D. Gravanis, G. Kakarontzas y V. C. Gerogiannis, «You don't need a Microservices Architecture (yet): Monoliths may do the trick,» en *Proceedings of the 2021 2nd European Symposium on Software Engineering (ESSE 2021)*, Xiamen, China, 2021. DOI: [10.1145/3501774.3501780](https://doi.org/10.1145/3501774.3501780).
- [4] V. Velepucha y P. Flores, «A Survey on Microservices Architecture: Principles, Patterns, and Migration Challenges,» *IEEE Access*, vol. 11, págs. 88 339-88 358, 2023. DOI: [10.1109/ACCESS.2023.3305687](https://doi.org/10.1109/ACCESS.2023.3305687).
- [5] T. Selivorstova, S. Klishch, S. Kyrychenko, A. Guda y K. Ostrovskaya, «Analysis of Monolithic and Microservice Architectures Features and Metrics,» *Computer Systems and Information Technologies*, n.º 3, págs. 59-65, 2022. DOI: [10.31891/CSIT-2021-5-8](https://doi.org/10.31891/CSIT-2021-5-8).
- [6] Y. Abgaz, A. McCarren, P. Elger, D. Solan, N. Lapuz y M. Bivol, «Decomposition of Monolith Applications Into Microservices Architectures: A Systematic Review,» *IEEE Transactions on Software Engineering*, vol. 49, n.º 8, págs. 4213-4242, 2023. DOI: [10.1109/TSE.2023.3287297](https://doi.org/10.1109/TSE.2023.3287297).
- [7] G. Blinowski, A. Ojdowska y A. Przybylek, «Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation,» *IEEE Access*, vol. 10, págs. 20 357-20 374, 2022. DOI: [10.1109/ACCESS.2022.3152803](https://doi.org/10.1109/ACCESS.2022.3152803).
- [8] P. Jatkiewicz y S. Okrój, «Differences in performance, scalability, and cost of using microservice and monolithic architecture,» en *Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing (SAC '23)*, New York, NY, USA, 2023. DOI: [10.1145/3555776.3578725](https://doi.org/10.1145/3555776.3578725).
- [9] M. Richards y N. Ford, *Fundamentals of Software Architecture: An Engineering Approach*. Sebastopol, CA, USA: O'Reilly Media, Inc., 2020, ISBN: 9781492043447.

- [10] S. Pinto-Agüero y R. Noel, «Microservices Evolution Factors: A Multivocal Literature Review,» *IEEE Access*, vol. 13, págs. 88 707-88 730, 2025. DOI: [10.1109/ACCESS.2025.3570658](https://doi.org/10.1109/ACCESS.2025.3570658).